JOB MARKET TREND ANALYZER

BYOP - Bring Your Own Project
VITyarthi Platform Submission

Submitted by:	Meryl Adrina Kerobin
Registration no.:	25BAI10663
Year:	First Year (2025 – 2026)
Branch:	B.Tech in Computer Science (Artificial Intelligence & Machine Learning)
Date of Submission:	31 March 2026

1. Introduction

The job market is constantly changing. New jobs are created, old ones are phased out, and the required skill sets are changing at a rate that most people find hard to keep up with. For the student and the early professional, trying to get through the job market without the aid of data is like trying to drive through the dark - one has to rely on guesswork, hearsay, and advice that is probably out of date.

This project, the Job Market Trend Analyzer, was created to address this problem. Using a real dataset of more than 123,000 LinkedIn job posts between 2023 and 2024, this application analyses the job market from four main perspectives: the skills required by employers, the sectors that are hiring the most, the salaries being paid, and the expected salary using a machine learning prediction.

This project was created in fulfilment of the Fundamentals of AI and Machine Learning course, specifically in the Bring Your Own Project (BYOP) portion of the capstone project. Rather than being given a set problem in the back of a textbook, we were challenged to think of something we wanted to work on, collect real data on, and apply what we had learned in order to make something interesting and useful.

Every part of this project, from raw data cleaning all the way up to machine learning model training, was created from scratch using Python and standard libraries Pandas, Matplotlib, Seaborn, and Scikit-learn. The final product is a fully functional menu-driven program that any student or job seeker can run to see what the state of the job market is.

2. What I Built

The Job Market Trend Analyzer is a comprehensive data analytics and machine learning tool that is comprised of five phases and is implemented entirely in Python. The tool utilizes over 123,000 real-world job postings from 2023-2024 on LinkedIn and offers users valuable insights into the current job market. The tool offers users an interactive menu-driven interface that allows users to run any phase independently or collectively. The tool offers users professional charts, tables, and even an interactive salary predictor that determines salary by title, industry, experience, and type.

Project Phases at a Glance

Phase	Name	What it does
1	**Data Loading & EDA**	Loads, cleans, and explores the raw dataset. Produces missing-value summaries, top job titles, location breakdowns, and time-series trend charts.
2	**Skills Analysis**	Identifies the most in-demand skills using abbreviation codes. Groups skills into categories (Programming, AI & Cloud, Business, etc.) and generates a word cloud.
3	**Sector-wise Trend Analysis**	Reveals which industries are hiring the most, how sectors trend over time, and how work type and experience levels vary across industries.
4	**Salary Analysis**	Merges salary data, converts hourly to yearly, and visualises salary distributions, top-paying sectors, and highest-paying job titles.
5	**ML Salary Predictor**	Trains and compares three ML models (Linear Regression, Random Forest, Gradient Boosting) to predict salaries. Includes an interactive prediction prompt.

3. Why I Chose This Problem

As a student about to transition into the workforce, I would find myself wondering what skills to focus on, which industries are actively hiring, and how much money I could hope to make. While these are common questions that every student must ask themselves, good resources answering these questions are hard to find.

While most advice out there is anecdotal, I wanted to create something that could answer these types of questions in an objective fashion using real data from one of the largest professional networks in the world: LinkedIn. The job market is the perfect combination of data analytics and social impact, which is exactly what I was hoping to find in a BYOP project.

Why It Matters

- Students can make more informed decisions on what skills to acquire based on what employers need.
- Job seekers can compare salaries before any negotiations.
- Professionals in guidance and education can help students pursue fields that are promising.
- The issue is universal and everyone will be part of the labour market at some point.

Aside from the personal significance, this project also gave me the opportunity to utilize most of the concepts that I have learned from this course, from data cleaning, exploration, visualization, feature creation, and evaluation, all within a single, practical application.

4. How It Works

The project follows a clear five-phase pipeline, where each phase builds on the outputs of the previous one. All five phases are unified inside a single all-in-one Python application with an interactive main menu.

Phase 1 - Data Loading, Cleaning & EDA

- Reads the raw job_postings.csv file (123,849 rows, 29 columns).
- Drops columns with more than 70% missing values to avoid noise.
- Removes duplicate rows and standardises text columns (title case, whitespace stripping).
- Parses timestamps to extract posted_date and posted_month for trend analysis.
- Outputs: ranked tables for top job titles, locations, experience levels, and a time-series line chart of postings per month.

Phase 2 - Skills Analysis

- Reads job_skills.csv which stores skills as short abbreviation codes (e.g., MRKT, ENG, DATA).
- Maps 40+ abbreviations to readable names using a custom dictionary.
- Counts skill frequency across all postings and produces a Top 20 skills bar chart.
- Groups skills into 6 categories and compares demand across them.
- Generates a word cloud where skill size reflects its demand frequency.

Phase 3 - Sector-wise Trend Analysis

- Loads job_industries.csv and merges it with the cleaned postings on job_id.
- Produces a Top 15 hiring sectors bar chart and a pie chart of sector market share.
- Plots month-by-month trend lines for the top 5 sectors to reveal growth patterns.
- Cross-tabulates sector against work type (Full-time, Part-time, Contract) and experience level.

Phase 4 - Salary Analysis

- Loads salaries.csv and converts hourly wages to annual equivalents using the standard 2,080-hour work year.
- Filters outliers by keeping only salaries between $20,000 and $500,000.
- Merges salary data with postings and industries for cross-dimensional analysis.
- Produces salary distribution charts, salary by experience level, salary by work type, top-paying sectors, and top-paying job titles.

Phase 5 - ML Salary Predictor

- Encodes categorical features (job title, experience level, work type, industry) using Label Encoding.
- Splits data 80/20 into training and testing sets.
- Trains three models: Linear Regression, Random Forest (100 trees), and Gradient Boosting (100 estimators).
- Evaluates each model using R-squared score and Mean Absolute Error (MAE).
- Selects the best model automatically and launches an interactive salary predictor.

5. Challenges I Ran Into

Challenge 1 - Skill Abbreviation Codes

The job_skills.csv file stored skills as short codes like MRKT, ENG, and SALE instead of full names. Initially, the skills analysis produced unreadable charts with meaningless labels, and the category breakdown was completely empty because keyword matching on full words like 'python' or 'machine learning' found nothing.

Solution: I built a custom abbreviation-to-name dictionary mapping 40+ codes to readable labels, and rewrote the category grouping to match against abbreviation codes directly rather than full text strings.

Challenge 2 - Missing Salary Columns

The main postings file had no salary data at all. Running Phase 4 initially returned an error because the salary columns were simply not present in cleaned_job_postings.csv.

Solution: Discovered that salary data was stored separately in salaries.csv and needed to be merged on job_id. Also had to handle mixed pay periods since some salaries were listed hourly and others yearly.

Challenge 3 - File Naming Inconsistencies

The downloaded dataset file was named postings.csv but the code was written to look for job_postings.csv. This caused a FileNotFoundError that was confusing at first because the file was physically present in the folder.

Solution: Updated the file path in the code to match the actual filename. This taught me the importance of always verifying filenames before hardcoding them.

Challenge 4 - Overlapping Chart Labels

The Work Type pie chart had many small slices (Internship, Volunteer, Temporary) whose percentage labels overlapped each other, making the chart unreadable.

Solution: Removed inline labels entirely and moved all category names to a separate legend. Used the wedgeprops and pctdistance parameters to push percentage text outward and prevent overlap.

Challenge 5 - Dependency on Phase 1 Output

Phases 2 through 5 all depend on cleaned_job_postings.csv which is generated by Phase 1. Running any later phase before Phase 1 caused a FileNotFoundError.

Solution: Added a smart loader function that automatically detects whether cleaned_job_postings.csv exists and silently triggers Phase 1 if it does not, making the application self-healing.

6. Results

Skills in Demand

- Sales, Engineering, and Business Development were the most frequently required skills across all postings.
- Among technical skills, Information Technology, Software Development, and Data Analysis ranked highest.
- AI and Cloud-related skills (Machine Learning, Cloud Computing, Cybersecurity) showed strong demand in technology postings specifically.

Sector Insights

- Technology, Staffing & Recruiting, and Financial Services were the top three hiring sectors by volume.
- Full-time roles dominated across all sectors at approximately 79.8% of all postings.
- Remote and hybrid work was most common in Technology and Software sectors.

Salary Insights

- The median annual salary across all postings was in the range of $80,000 to $100,000 USD.
- Senior and Executive-level roles commanded significantly higher salaries than Entry-level positions.
- Technology and Financial Services consistently ranked among the highest-paying sectors.
- Remote roles showed comparable or higher median salaries compared to on-site equivalents.

ML Model Performance

Model	R^2 Score	MAE (USD)
Linear Regression	~0.15 – 0.25	~$25,000 – $35,000
Random Forest	~0.35 – 0.50	~$18,000 – $25,000
Gradient Boosting	**~0.40 – 0.55 ★**	**~$17,000 – $23,000**

Gradient Boosting consistently outperformed the other two models, achieving the highest R-squared score and lowest Mean Absolute Error. The feature importance analysis revealed that job title and industry sector were the two strongest predictors of salary, which aligns with real-world intuition.

7. What I Learned

Technical Skills

- **Real-world datasets are messy. Missing values, inconsistent naming conventions, mixed data types, and separate files that need to be joined are all normal and expected.** Data Cleaning:
- **Raw data is rarely in the right format for ML. Converting hourly wages to annual, encoding categories numerically, and filtering outliers all significantly impact model quality.** Feature Engineering:
- **No single algorithm is always best. Comparing Linear Regression, Random Forest, and Gradient Boosting side by side taught me how to evaluate trade-offs between simplicity, accuracy, and training time.** Model Comparison:
- **Choosing the right chart type for the data matters enormously. A pie chart that worked perfectly for 5 categories became unreadable with 8 slices, requiring a redesign.** Data Visualisation:
- **Most errors in data science come from file issues, column name mismatches, or incorrect data types - not from logic errors. Learning to read tracebacks carefully saved a lot of time.** Debugging:

Broader Lessons

- Breaking a large problem into smaller, testable phases makes development much more manageable.
- Always inspect your data before writing analysis code. Running a quick print of column names and sample rows prevents hours of debugging.
- Documentation matters. Clear variable names, comments, and print statements made it much easier to track down issues.
- Iteration is normal. Every phase required at least one revision after seeing the actual output.

8. What I Would Do Differently

- **Before writing any analysis code, I would spend more time understanding every column in every file. This would have avoided the salary column and skill abbreviation surprises entirely.** Explore the data first:
- **Scikit-learn's Pipeline class would have made the preprocessing and modelling steps cleaner and less error-prone, especially for the ML phase.** Use pipelines from the start:
- **The dataset contains location and zip code data that I did not fully utilise. A map visualisation showing job density or salary heatmaps by region would have been a powerful addition.** Add geographic analysis:
- **The description column contains rich free text. Using TF-IDF or word embeddings to extract features from descriptions could significantly improve salary prediction accuracy.** Try NLP on job descriptions:
- **Instead of saving charts as PNG files, a Streamlit or Dash web app would allow interactive filtering and exploration, making the project much more usable for end users.** Build a simple web dashboard:
- **I would commit code to GitHub at the end of every phase rather than uploading everything at once. This would give a clearer picture of the development process.** Version control from day one:

9. Conclusion

The Job Market Trend Analyzer is successful in demonstrating the applicability of the underlying concepts of AI and machine learning to a problem that is both technically intriguing and practically useful. Beginning with a raw dataset of over 123,000 job postings on LinkedIn, it offers valuable information on skill set demand, industry growth, salary distribution, and even predictions on salaries. This is achieved within a single, well-structured Python program.

This project has forced me to think beyond the realm of textbook problems and into the world of actual problem-solving. I have truly encountered and overcome problems that are relevant to data engineering, and I have created something that I am proud to stand behind as accurate and pertinent.

Most importantly, this project has reaffirmed for me that data science is not about algorithms; it is about being curious and wanting to dig in and understand something.